Hands-On Lab

Create ECR, Install Docker, Create Image, and Push Image To ECR

Test this lab to learn how to push Docker images to AWS ECR. Ideal for DevOps Engineers and Cloud Developers, this lab covers creating Docker images, pushing them to ECR, and integrating with AWS services for efficient container management and deployment.

Scenario:

Netflix, a global leader in video streaming, has adopted a microservices architecture to support its highly scalable platform. To efficiently manage and deploy these microservices, Netflix utilizes Amazon EKS for orchestrating containers and Amazon ECR as a secure and scalable Docker container registry. Developers at Netflix create Docker images locally for their microservices, push them to ECR, and deploy them on Amazon EKS clusters. This streamlined CI/CD pipeline ensures that only authorized users can manage images, maintaining security and ensuring efficient lifecycle management for microservices across a global infrastructure.

Description:

This hands-on lab guide walks through the process of setting up a private Docker container registry using Amazon Elastic Container Registry (ECR) for a microservices-based application hosted on an Amazon EKS cluster. It provides detailed steps for creating an ECR repository, installing Docker, building Docker images locally, and pushing these images to ECR. This secure and scalable workflow ensures that only authorized users can push or pull images, maintaining the integrity and security of the microservices deployment.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

K21Academy
Learn Cloud, AI & ML from Experts

Contents

1 Introduction **3**
2 Overview & Key Points **4**
2.1 Container 4
2.2 Docker 4
2.3 Push & Pull 5
3 Documentation Links **6**
4 Prerequisite **7**
5 Cost To Perform This Lab **8**
6 Creating a Repository in ECR **9**
7 Install & Configure AWS CLI **12**
7.1 Configure AWS CLI 13
8 Install Docker & Create Image **16**
8.1 Installing Docker On an EC2 Instance 16
8.2 Creating a Docker File and building an Image from it 19
9 Push Docker Image to ECR **22**
10 Share your learnings **26**
10.1 On LinkedIn 27
10.2 Share the win with the Community 31
11 Cleanup Resources **32**
11.1 Delete the pushed image and ECR 32
12 Troubleshooting **34**
12.1 Unable to locate credentials 35
12.1.1 Issue 35
12.1.2 Cause 35
12.1.3 Fix 35
13 Summary **36**

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

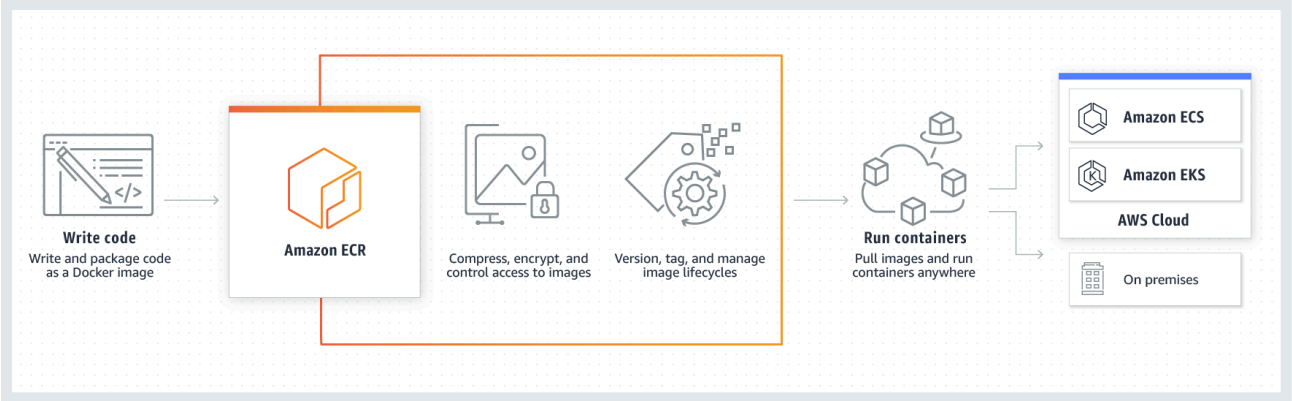
1 INTRODUCTION

ECR: Amazon Elastic Container Registry (ECR) is a fully managed Docker container registry that makes it easy for developers to store, manage, and deploy Docker container images.

Amazon ECR is integrated with Amazon Elastic Container Service (ECS), simplifying your development to production workflow. Amazon ECR eliminates the need to operate your own container repositories or worry about scaling the underlying infrastructure.

Amazon ECR hosts your images in a highly available and scalable architecture, allowing you to reliably deploy containers for your applications.

Integration with AWS Identity and Access Management (IAM) provides resource-level control of each repository. With Amazon ECR, there are no upfront fees or commitments. You pay only for the amount of data you store in your repositories and data transferred to the Internet.



This activity guide covers steps for:

1. Creating a Repository in ECR
2. Configure AWS credentials and install Docker
3. Creating a Docker file and building Images from it
4. Pushing images to the Repository

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

2 Overview & Key Points

In this Section, we will be covering a few of the services and terminologies we are using in the guide.

2.1 Container

A container is a lightweight, portable, and executable software package that includes everything needed to run a piece of software, including the code, runtime, libraries, and system tools. Containers provide consistency across different environments, making it easier to develop, deploy, and scale applications.

AWS ECR (Elastic Container Registry)

AWS ECR is a fully managed container registry service that makes it easy for developers to store, manage, and deploy Docker container images. It integrates seamlessly with other AWS services, providing a secure and scalable solution for storing and deploying containerized applications.

EKS Cluster (Amazon EKS)

Amazon EKS (Elastic Kubernetes Service) is a managed Kubernetes service that simplifies the deployment, management, and scaling of containerized applications using Kubernetes on AWS. It eliminates the need for you to install and operate your own Kubernetes clusters.
https://k21academy.com/docker-kubernetes/amazon-eks-kubernetes-on-aws/

2.2 Docker

Docker is a platform for developing, shipping, and running applications in containers. It simplifies the process of creating and managing containers, allowing developers to package their applications and all dependencies into a single unit for easy deployment and execution.

Docker Image

A Docker image is a lightweight, standalone, and executable package that includes the application code, runtime, libraries, and other dependencies required to run an application. Images are used to create containers when executed.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

2.3 Push & Pull

Push & Pull (Docker)

Push: In Docker, pushing refers to the process of uploading a Docker image to a container registry, making it available for others to pull and use.

Pull: Pulling in Docker involves downloading a Docker image from a container registry to your local system or another environment.

Git Push & Pull

Push (Git): Pushing in Git involves uploading your local changes (commits) to a remote repository, making them accessible to others collaborating on the same project.

Pull (Git): Pulling in Git is the process of fetching changes from a remote repository and merging them into your local branch, ensuring your code is up-to-date with the latest changes.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

3 Documentation Links

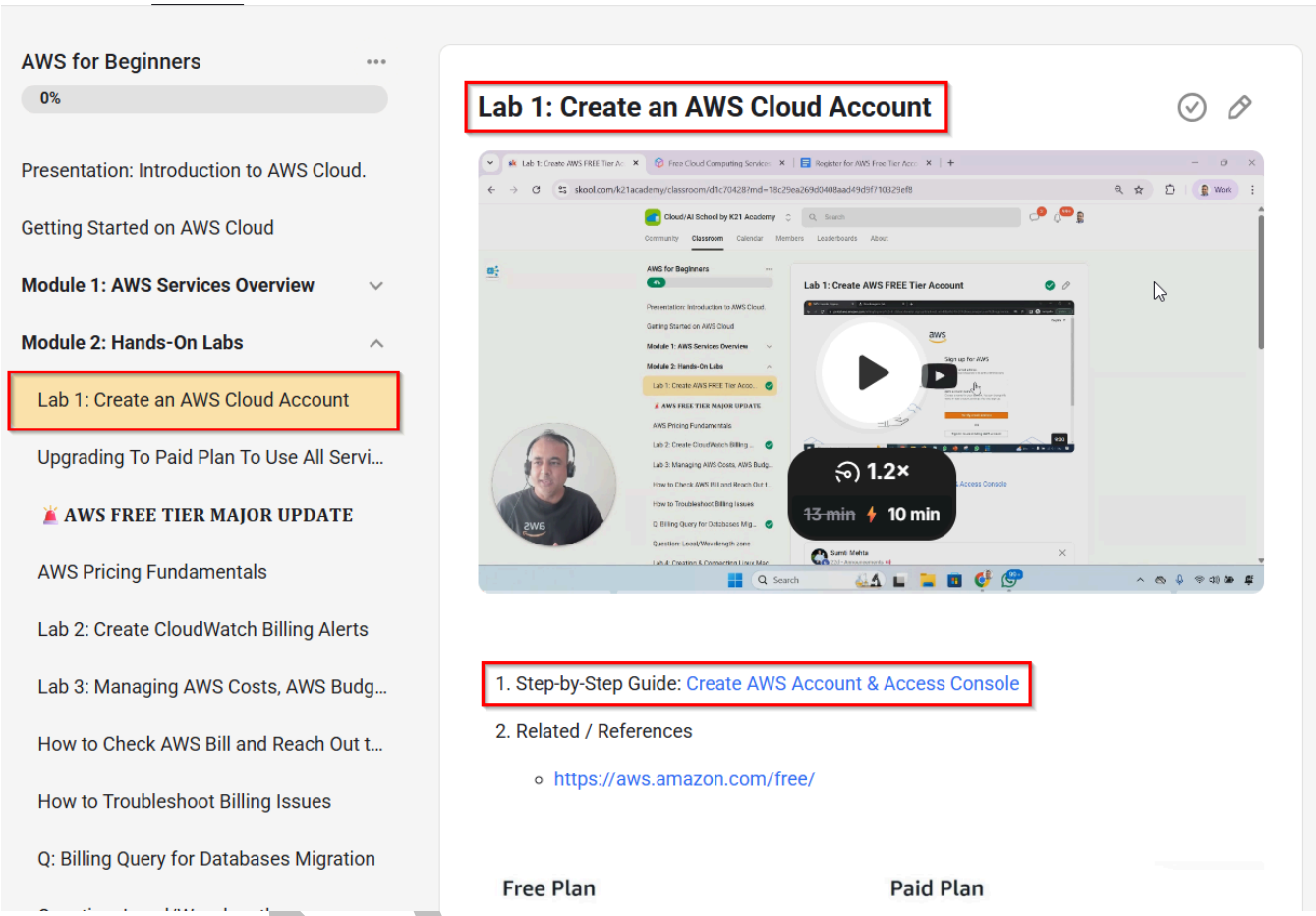
1. Create a Repository using the AWS Console
   https://docs.aws.amazon.com/AmazonECR/latest/userguide/repository-create.html
2. Create a Repository using CLI
   https://docs.aws.amazon.com/cli/latest/reference/ecr/create-repository.html
3. Amazon ECR
   https://aws.amazon.com/ecr/
4. Amazon ECR FAQ's
   https://aws.amazon.com/ecr/faqs/
5. Docker basics for Amazon ECS and ECR
   https://docs.aws.amazon.com/AmazonECS/latest/developerguide/docker-basics.html

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

4 PREREQUISITE

1. You must have an AWS Account. [Check out the Step-by-step Guide Here]



***Note:** In this guide, we are going to use a Linux machine to perform this lab. You can provision a Linux machine on Amazon.*

Follow *Activity Guide Create_&_Connect_To_EC2_Linux_Instance from the Portal* to create Linux machines on AWS. Click Here

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

5 Cost To Perform This Lab

Total Estimated Cost for 3-5 Hours:

- **For 3 hours:** Approx. **$0**
- **For 5 hours:** Approx. **$0**

Estimated Costs for 3-5 Hours of the Lab:

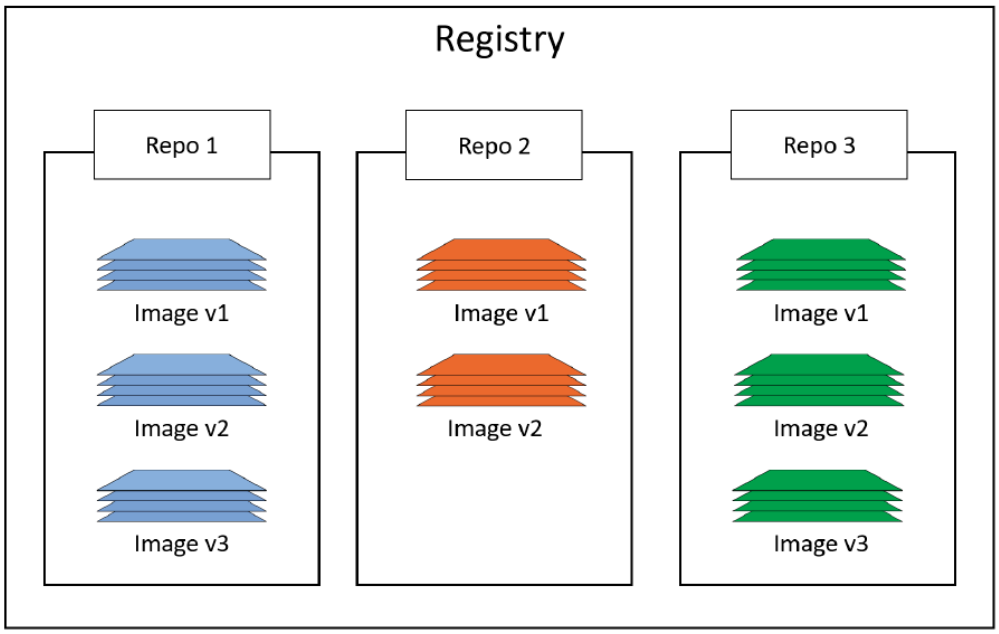
1. AWS EC2 Instance:
   - Instance Type: **t2.micro** (Covered under AWS Free Tier)
   - Cost: $0

2. Amazon ECR (Elastic Container Registry):
   - Storage: 500 MB free per month for private repositories under AWS Free Tier
   - Cost: $0

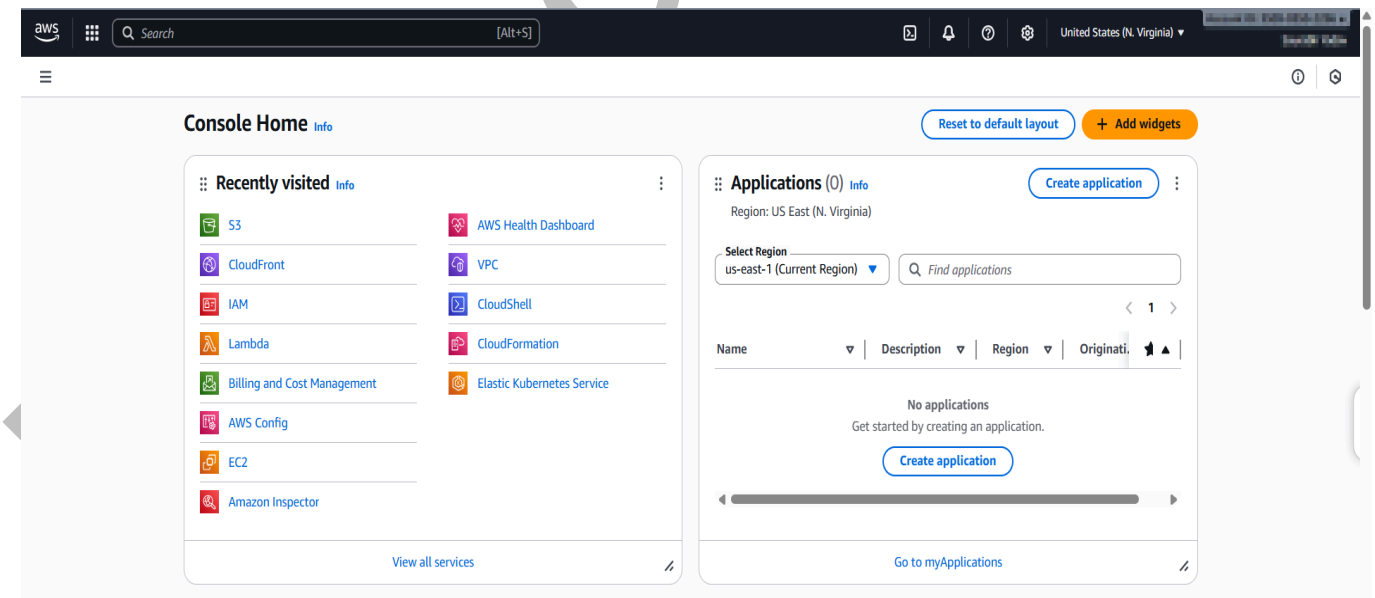
K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

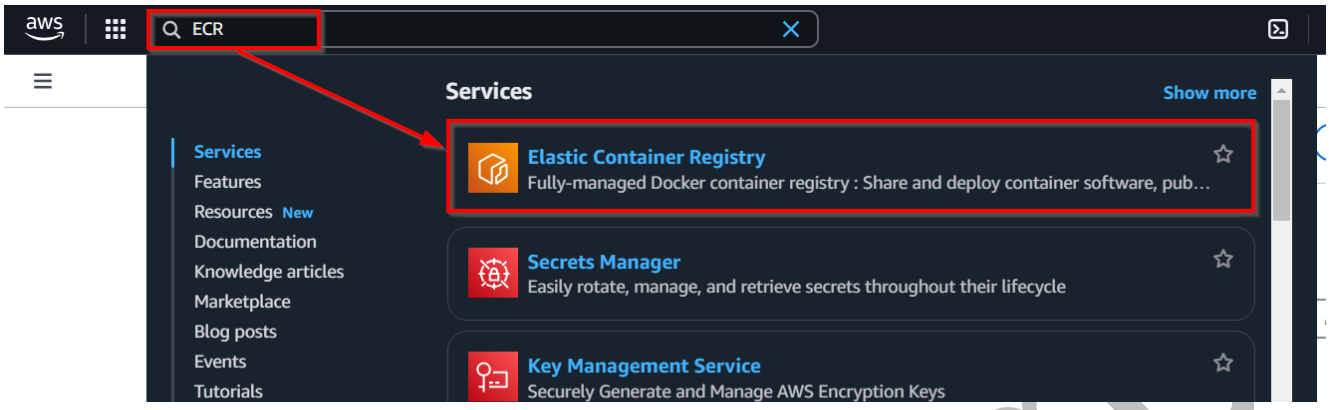
6 CREATING A REPOSITORY IN ECR



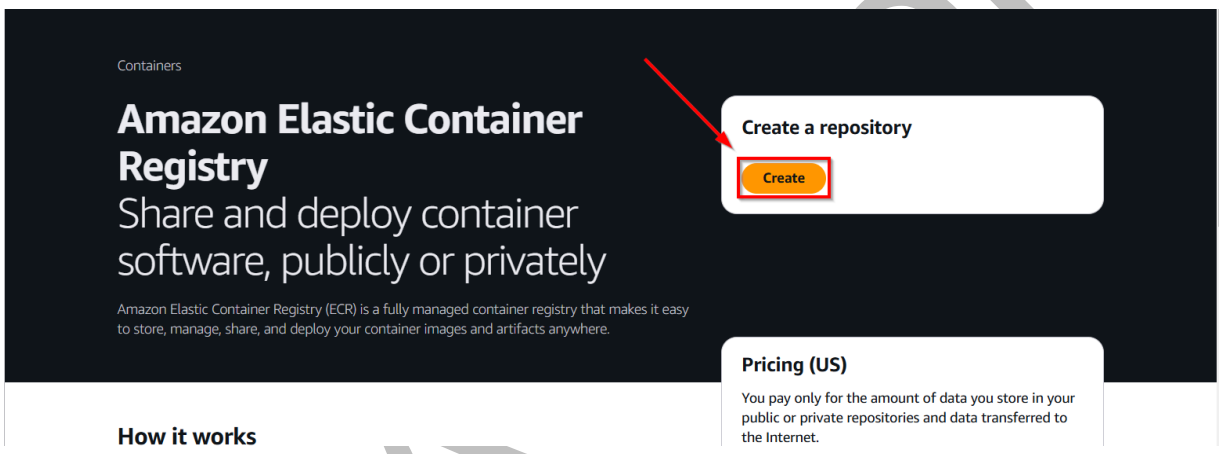
1. Open your Console and search ECR.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



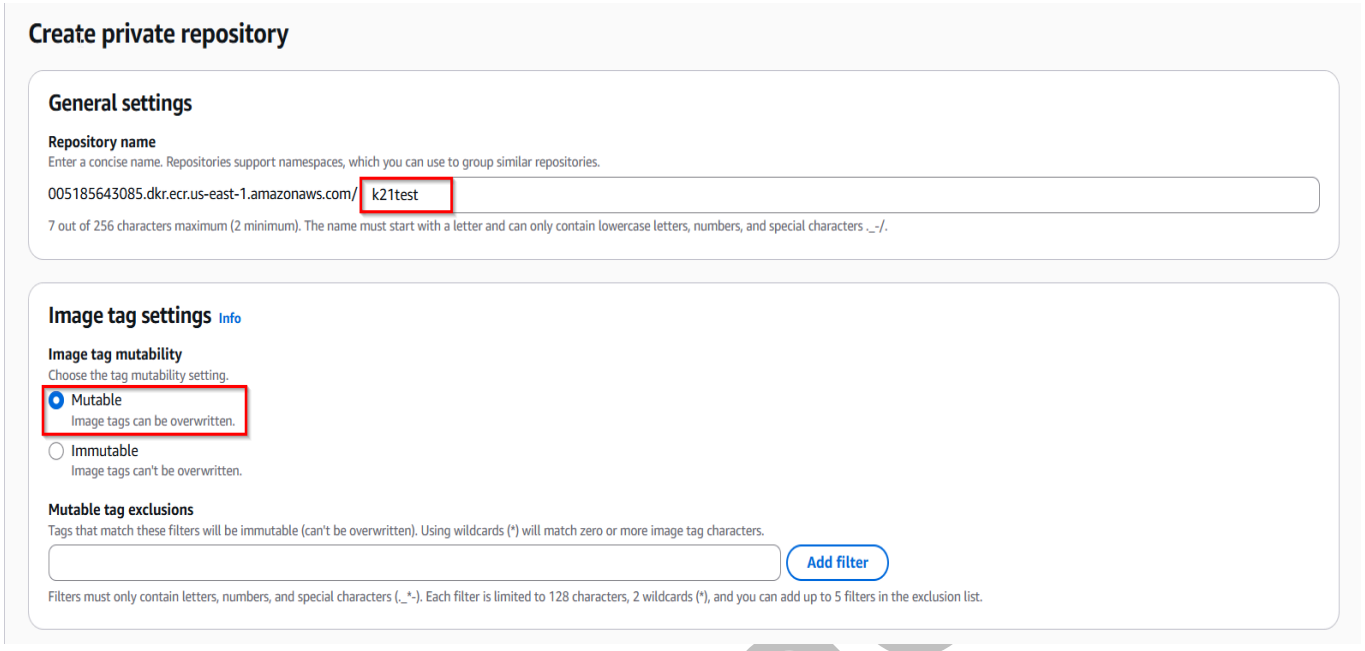
2. Click on Create Repository.

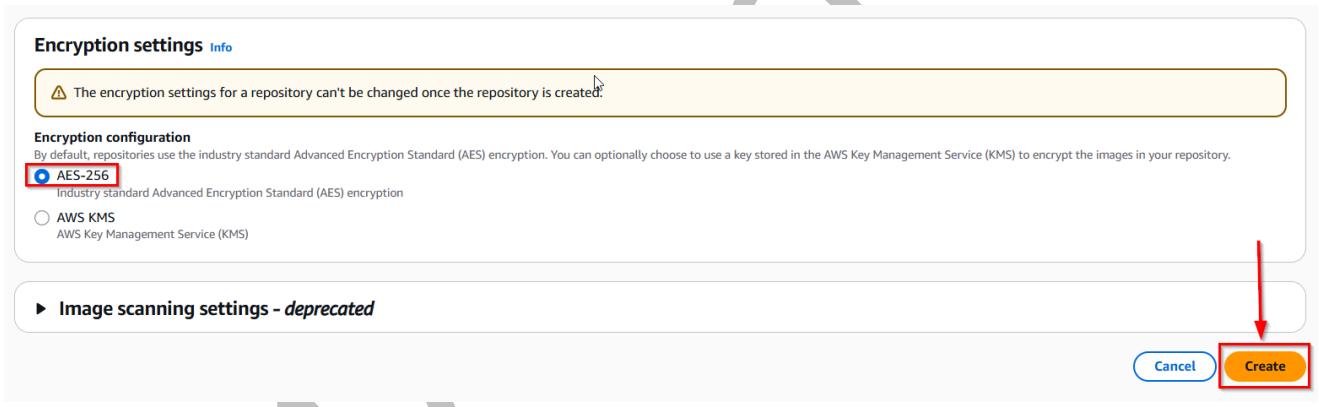


3. Enter your repository name, keep the settings according to your needs, and click on Create Repository.

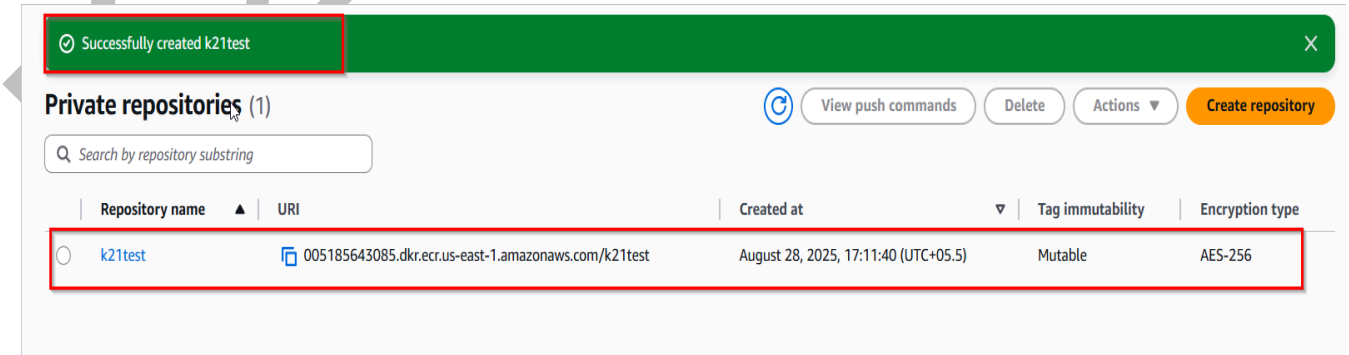
K21A

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission





4. Repository Created Successfully.



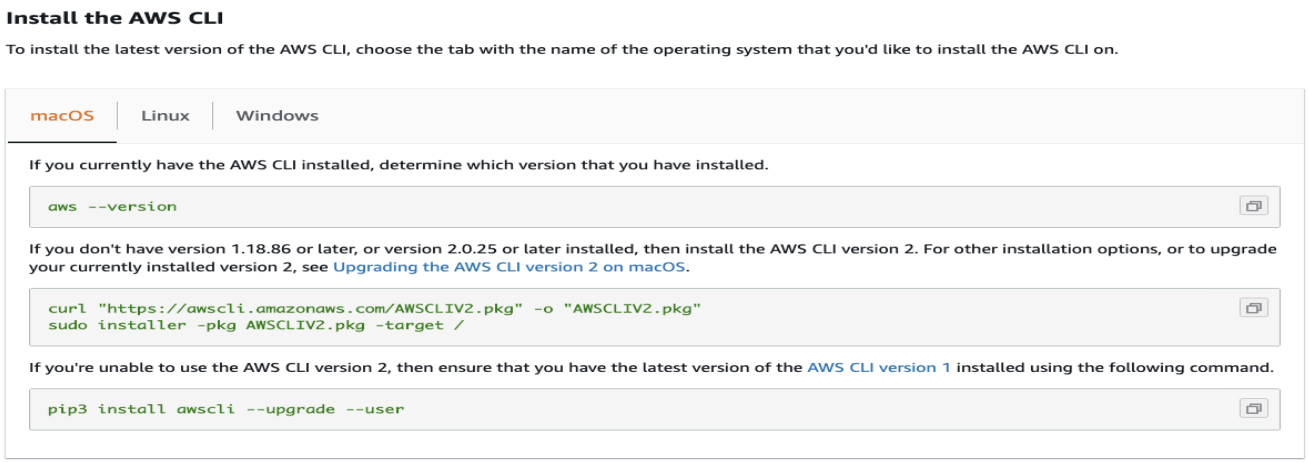
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

7 INSTALL & CONFIGURE AWS CLI

Note: *Steps in this section are optional and only required if you are using any Linux machine apart from Amazon Linux. Amazon Linux comes pre-installed with* ***AWS CLI,*** *so if you are using* ***Amazon Linux****, you can ignore the steps in this section and move to the next, configure AWS CLI*

Follow the links and snapshots to install and configure AWS CLI on Linux, Windows, and macOS

- https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-quickstart.html



1. For the latest version of the AWS CLI on Linux, use the following command block:

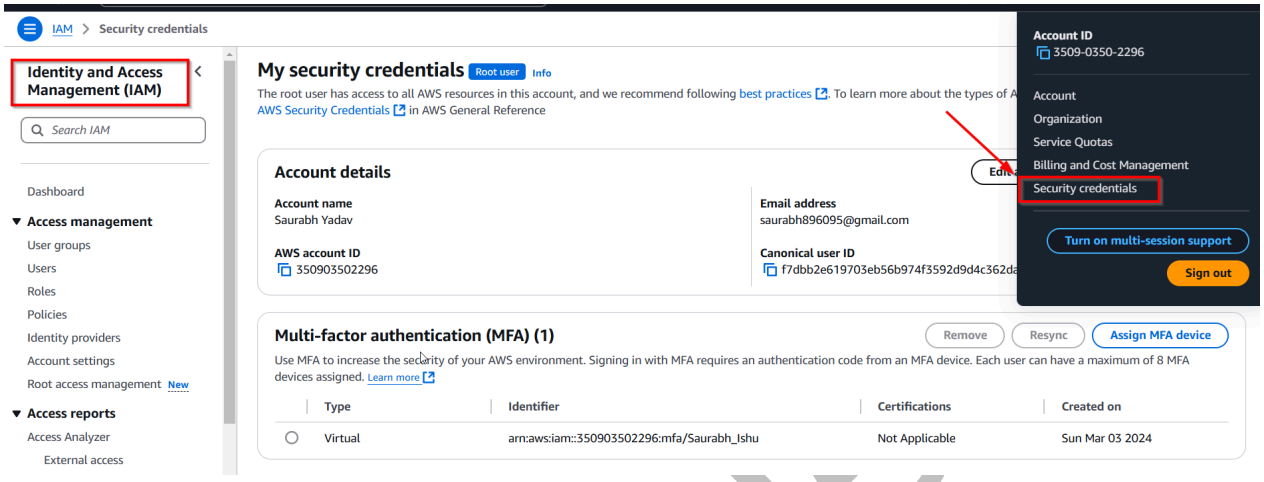
```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ sudo apt install unzip #(Ubuntu)

$ sudo yum install unzip #(Linux)
$ unzip awscliv2.zip
$ sudo ./aws/install
$ aws --version
aws-cli/2.0.60 Python/3.7.3 Linux/5.4.0-1031-azure exe/x86_64.ubuntu.18
```

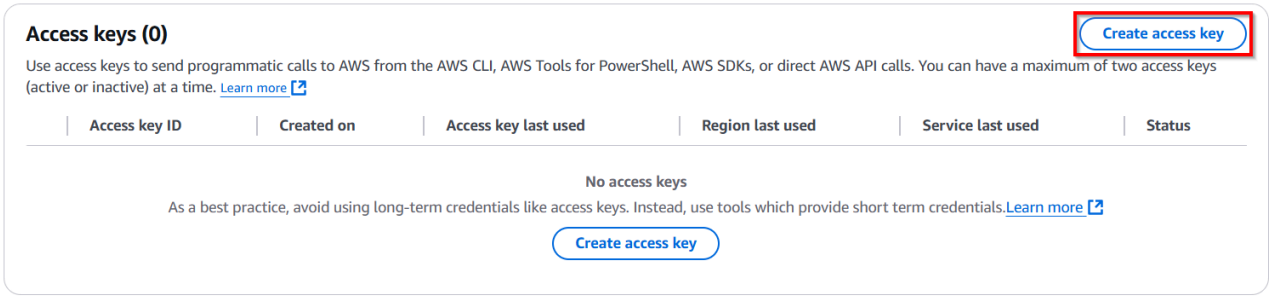
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

7.1 Configure AWS CLI

1. Create IAM Access Key



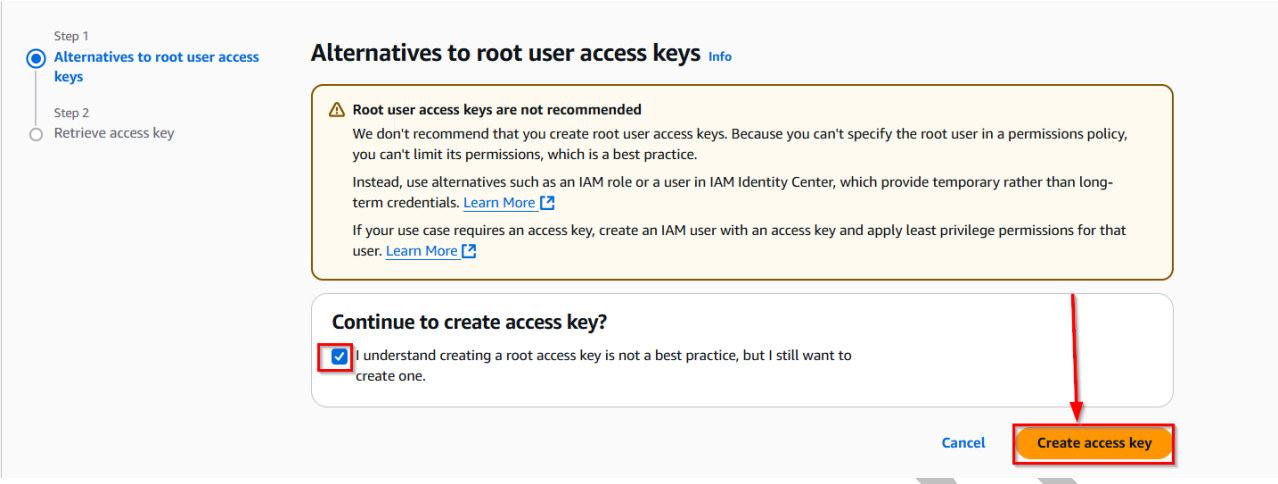
2. Scroll down to the Access key and select Create Access keys



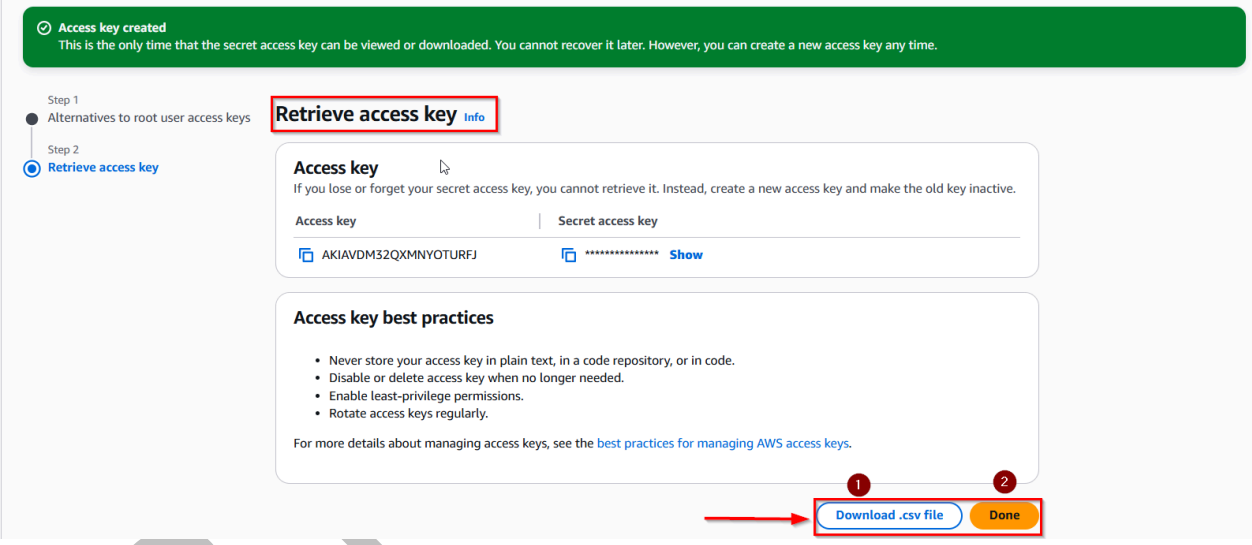
Note: This Lab is performed as a Root User.

3. Click on Create Access Key.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

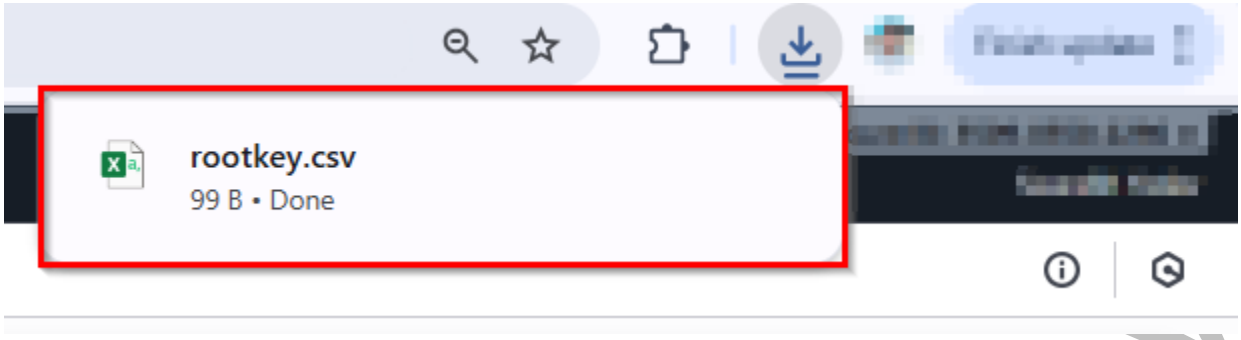


4. Download the .csv file and click on Done.



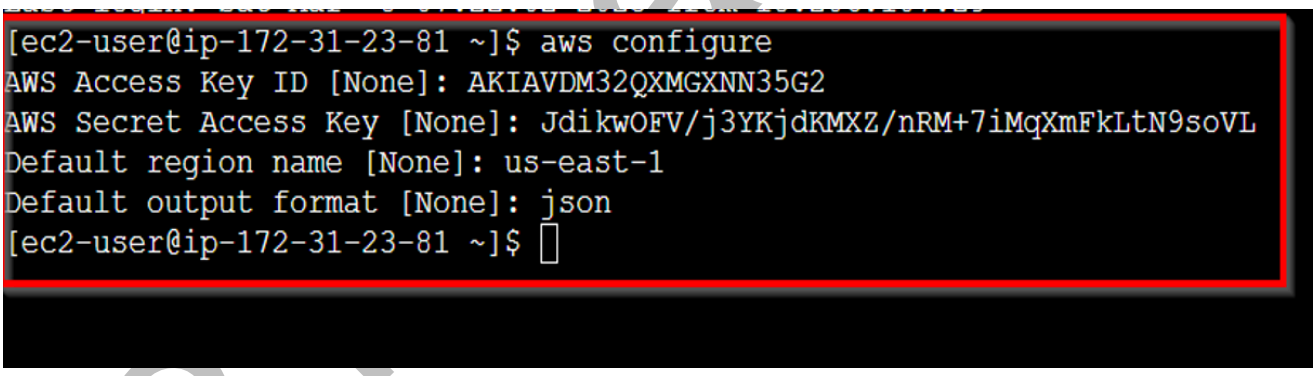
5. Now you can check the downloaded .**CSV** file here:

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



6. **Go back to your EC2** and Configure AWS CLI credentials:

```
$ aws configure
AWS Access Key ID [None]: <AKIAIOSFODNN7EXAMPLE>
AWS Secret Access Key [None]: <wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY>
Default region name [None]: <region-code>
Default output format [None]: <json>
```



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

8 Install Docker & Create Image

Docker: It's a platform for developing, shipping, and running applications in containers. Containers allow you to package an application and its dependencies together, ensuring consistency across different environments.

In Docker, an image is a lightweight, standalone, and executable package that includes everything needed to run a piece of software, including the code, runtime, libraries, and system tools. Creating an image involves defining a set of instructions (Dockerfile) that specify how the image should be built.

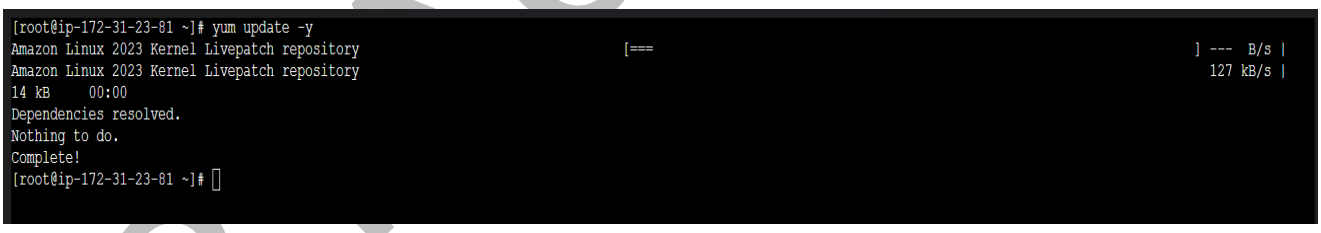
8.1 Installing Docker On an EC2 Instance

1. Switch to the root user

```
$ sudo -i
```

2. First Update Software Repositories

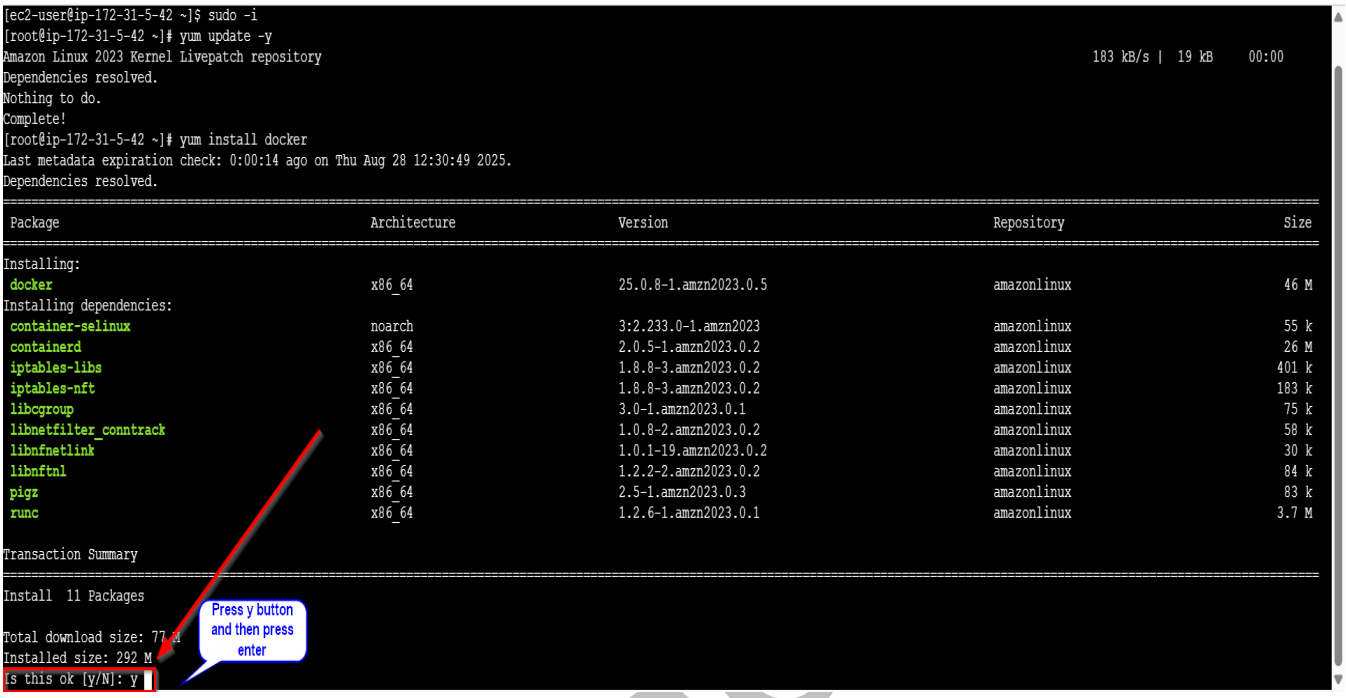
```
$ yum update -y
```

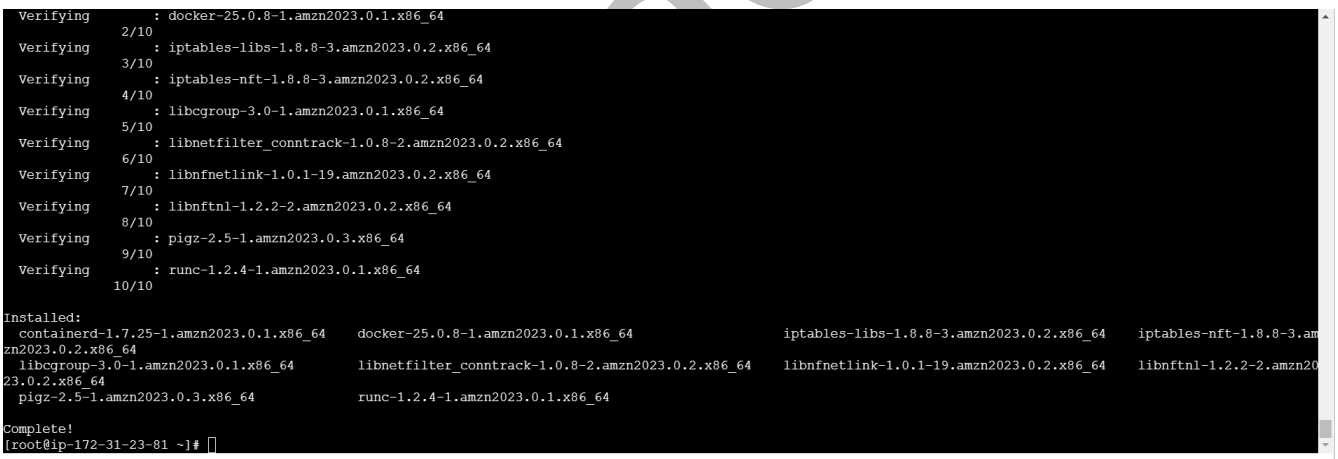


3. Install Docker

```
$ yum install docker
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission





4. Start and Enable Docker

```
$ systemctl start docker
$ systemctl enable docker
```

5. Check Docker status

```
$ systemctl status docker
```

You should see output like below (exit by typing q)

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
[root@ip-172-31-23-81 ~]# systemctl start docker
[root@ip-172-31-23-81 ~]# systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
     Active: active (running) since Sat 2025-03-08 07:43:44 UTC; 1min 3s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 28352 (dockerd)
      Tasks: 7
     Memory: 30.0M
        CPU: 263ms
     CGroup: /system.slice/docker.service
             └─28352 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

Mar 08 07:43:43 ip-172-31-23-81.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Mar 08 07:43:43 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:43.494550101Z" level=info msg="Starting up"
Mar 08 07:43:43 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:43.553155455Z" level=info msg="Loading containers: start."
Mar 08 07:43:43 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:43.985310482Z" level=info msg="Loading containers: done."
Mar 08 07:43:44 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:44.007086300Z" level=info msg="Docker daemon" commit=71907ca containerd-snapshotter
=false storage-driv
Mar 08 07:43:44 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:44.007338420Z" level=info msg="Daemon has completed initialization"
Mar 08 07:43:44 ip-172-31-23-81.ec2.internal dockerd[28352]: time="2025-03-08T07:43:44.045901155Z" level=info msg="API listen on /run/docker.sock"
Mar 08 07:43:44 ip-172-31-23-81.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
```

6. Check Docker Version

```
$ docker --version
```

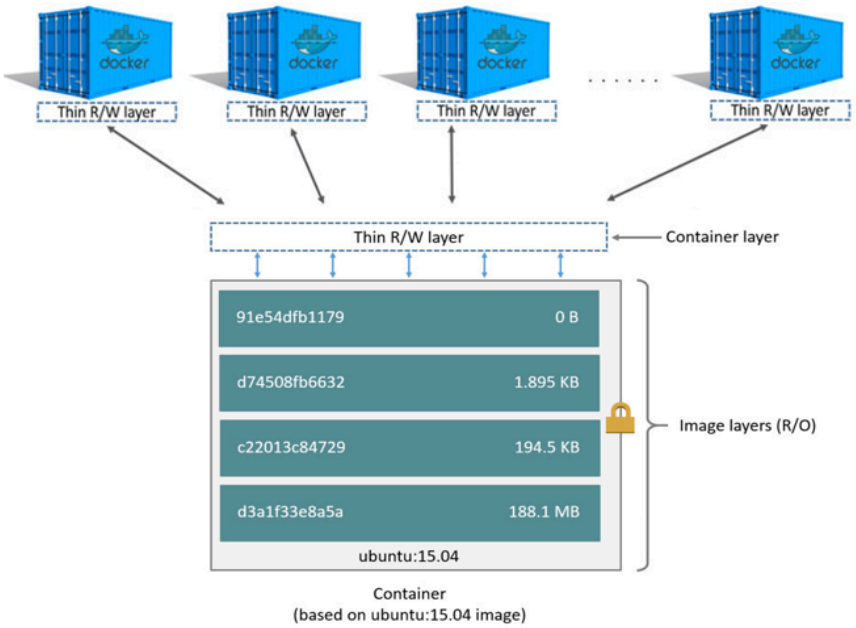
You should see output like

```
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]# docker --version
Docker version 25.0.8, build 0bab007
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
```

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

8.2 Creating a Docker File and building an Image from it



1. Download the Dockerfile from GitHub by running the command below

```
$ wget https://raw.githubusercontent.com/k21academyuk/Kubernetes/master/ECS/Dockerfile
$ ls
```

```
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]# wget https://raw.githubusercontent.com/k21academyuk/Kubernetes/master/ECS/Dockerfile
--2025-03-08 07:48:26--  https://raw.githubusercontent.com/k21academyuk/Kubernetes/master/ECS/Dockerfile
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.110.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 536 [text/plain]
Saving to: 'Dockerfile'

Dockerfile                                          0%[                                                                                                    ]
ckerfile                                          100%[==========================================================================================================>]
    in 0s

2025-03-08 07:48:26 (41.7 MB/s) - 'Dockerfile' saved [536/536]

[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
[root@ip-172-31-5-42 ~]# ls
Dockerfile
[root@ip-172-31-5-42 ~]#
```

2. View the content of the file.

```
$ cat Dockerfile
```

```
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]# cat Dockerfile
FROM ubuntu:18.04

# Install dependencies
RUN apt-get update && \
 apt-get -y install apache2

# Install apache and write hello world message
RUN echo 'Hello World!' > /var/www/html/index.html

# Configure apache
RUN echo '. /etc/apache2/envvars' > /root/run_apache.sh && \
 echo 'mkdir -p /var/run/apache2' >> /root/run_apache.sh && \
 echo 'mkdir -p /var/lock/apache2' >> /root/run_apache.sh && \
 echo '/usr/sbin/apache2 -D FOREGROUND' >> /root/run_apache.sh && \
 chmod 755 /root/run_apache.sh

EXPOSE 80

CMD /root/run_apache.sh
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
```

3. Now build the Docker image from the Dockerfile.

```
docker build -t hello-world .
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
-]# docker build -t hello-world .

ild definition from Dockerfile
kerfile: 635B
adata for docker.io/library/ubuntu:18.04
ckerignore
text: 2B
io/library/ubuntu:18.04@sha256:152dc042452c496007f07ca9127571cb9c29697f42acbfad72324b2bb2e43c98
io/library/ubuntu:18.04@sha256:152dc042452c496007f07ca9127571cb9c29697f42acbfad72324b2bb2e43c98
52c496007f07ca9127571cb9c29697f42acbfad72324b2bb2e43c98 1.33kB / 1.33kB
63a7ba4c1f0e710986f5a25e672842963d95b960191e2d9f7185ebe 424B / 424B
492e823bf5d51f1bd5f87ea3eed1cb31788686aa99a2fb61a27af6a 2.30kB / 2.30kB
c7634afb95a0fb2410a74b7b5bc0ba527033362c240c7a11bef4331 25.69MB / 25.69MB
6:7c457f213c7634afb95a0fb2410a74b7b5bc0ba527033362c240c7a11bef4331
```

```
Step 3/6 : RUN echo 'Hello World!' > /var/www/html/index.html
 ---> Running in dc458cad54b2
Removing intermediate container dc458cad54b2
 ---> 0e42d88e48be
Step 4/6 : RUN echo '. /etc/apache2/envvars' > /root/run_apache.sh &&  echo 'mkdir -p /var/run/apache
apache2' >> /root/run_apache.sh &&  echo '/usr/sbin/apache2 -D FOREGROUND' >> /root/run_apache.sh &&
 ---> Running in 4a076c6bf1b7
Removing intermediate container 4a076c6bf1b7
 ---> 8e78c7047330
Step 5/6 : EXPOSE 80
 ---> Running in dd94e85024a8
Removing intermediate container dd94e85024a8
 ---> 9ad6a0abf459
Step 6/6 : CMD /root/run_apache.sh
 ---> Running in 560d9ca0c002
Removing intermediate container 560d9ca0c002
 ---> 9b0bb6fc5554
Successfully built 9b0bb6fc5554
Successfully tagged hello-world:latest
```

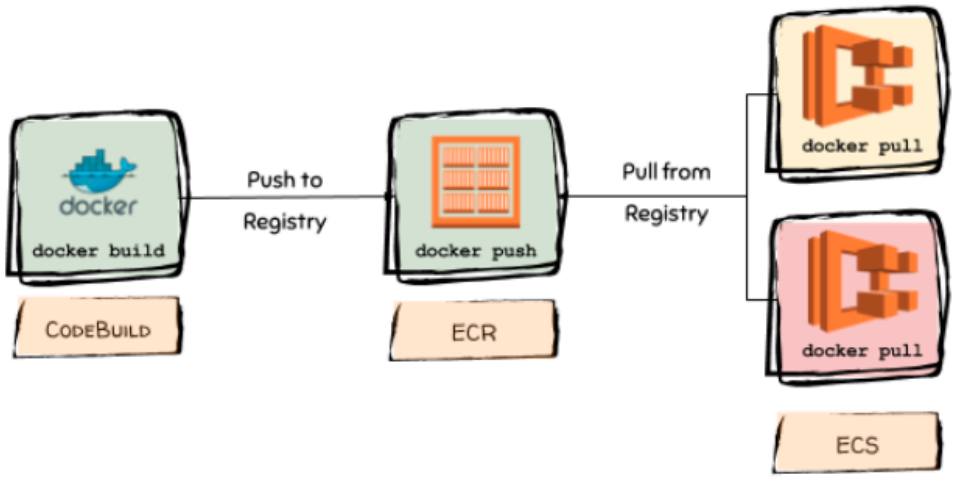
4. Validate Docker images created.

```
docker images
```

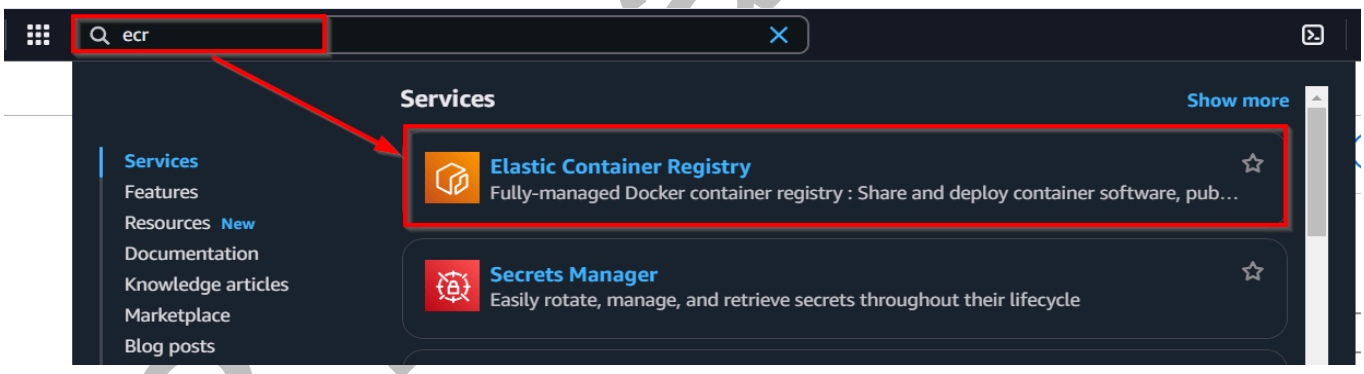
```
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
hello-world   latest    1fbe6bdf4d5f   6 minutes ago    205MB
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
[root@ip-172-31-23-81 ~]#
```

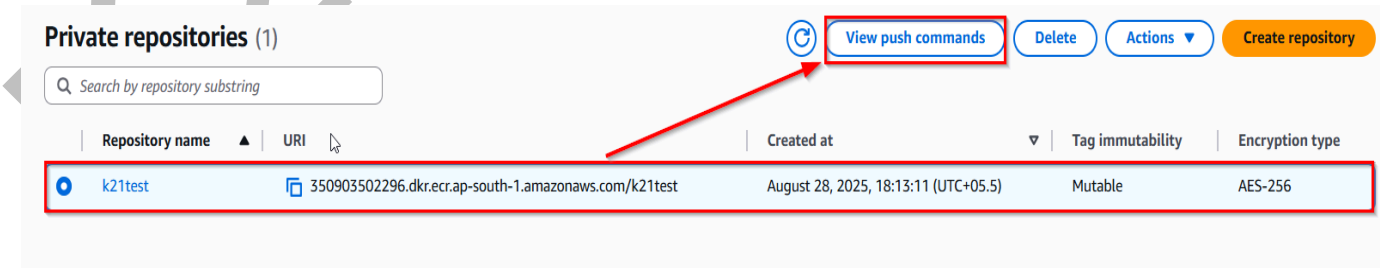
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

9 Push Docker Image to ECR

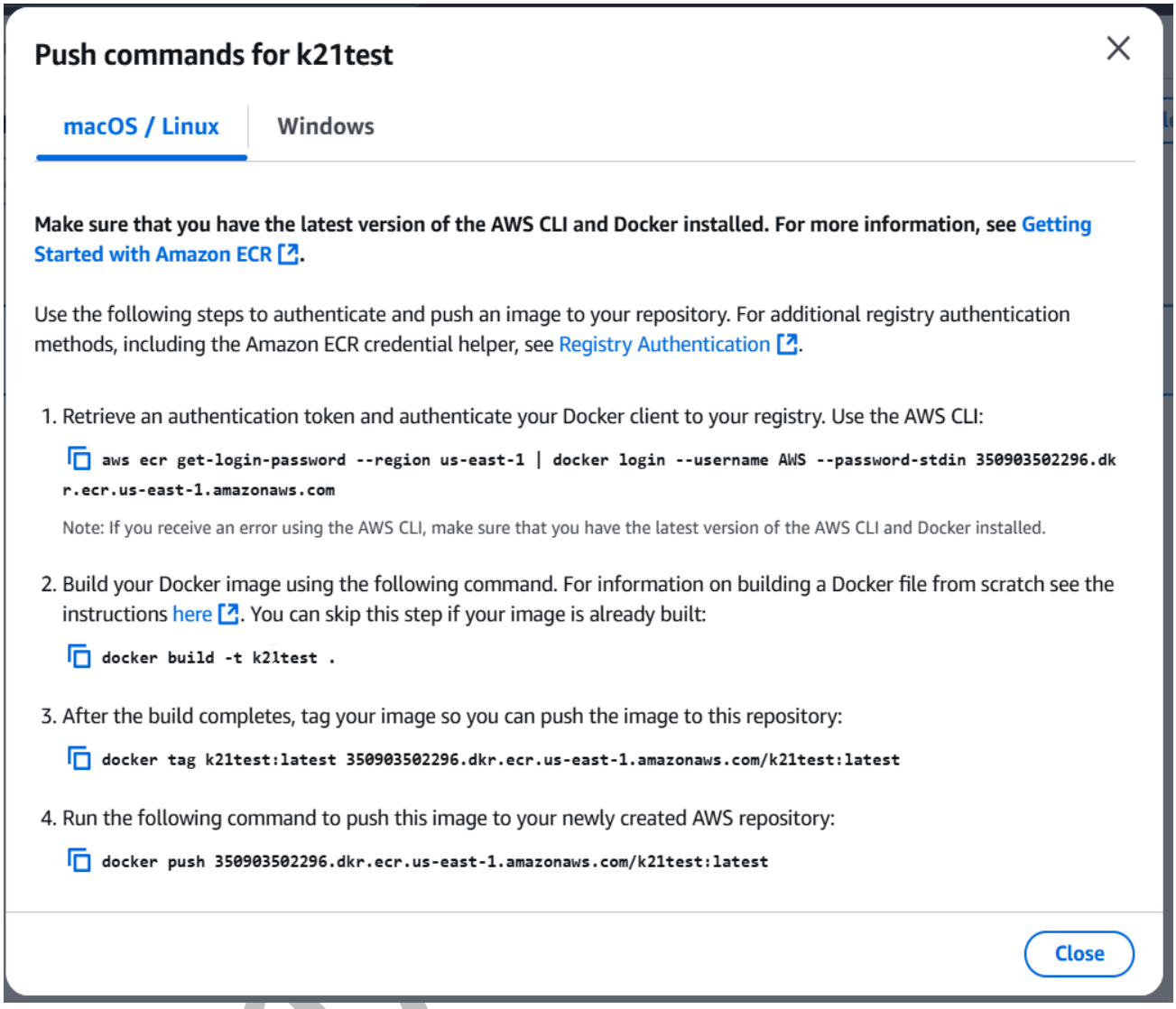


1.Go to the ECR console, then access the repo, and click on **View push commands**

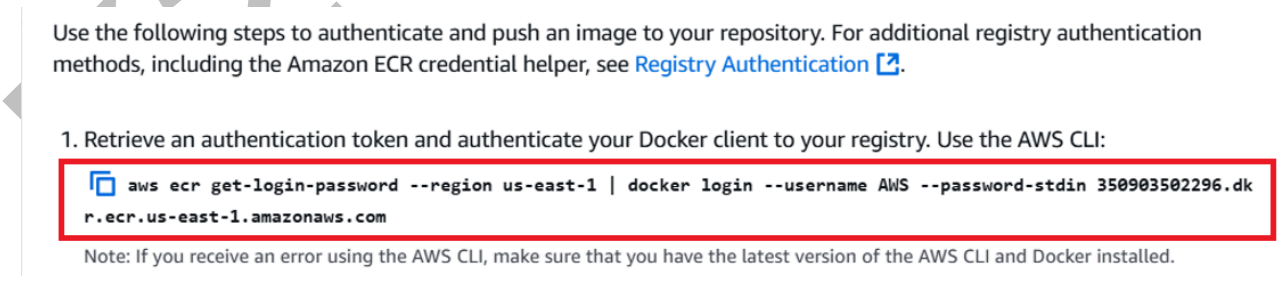




Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



2.Copy the get-login password command and run



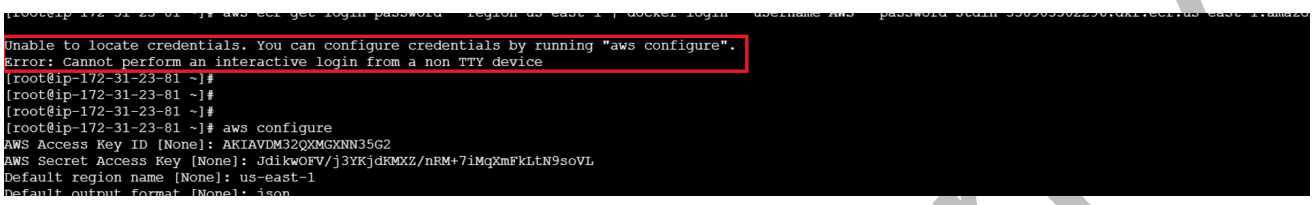


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

Note: If you get an error like "***Unable to locate credentials***. *You can configure credentials by running 'aws configure". Error: Cannot perform an interactive login from a non-TTY device.*"

Then make sure you follow "***aws configure***" from the guide.

Kindly refer to the troubleshooting section 12.1



3.Copy & run the **Docker build** command,

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions here. You can skip this step if your image is already built:

```
docker build -t k21test .
```

`-t k21test`: The `-t` flag is used to tag the image with a specific name, which in this case is `k21test`. This makes it easier to refer to the image later without using an autogenerated ID. This command builds a Docker image from a Dockerfile in the current directory and tags it as `k21test`

4.Copy the **Docker tag** command and change the image name, tag then run by following the instructions below.

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag k21test:latest 350903502296.dkr.ecr.us-east-1.amazonaws.com/k21test:latest
```

Tag your image with the Amazon ECR registry, repository, and image tag name combination to use.

Note: The registry format is aws_account_id.dkr.ecr.region.amazonaws.com. The repository name should match the repository that you created for your image.

```
docker tag <Image_Name>:<image tag>
<aws_account_id>.dkr.ecr.<region>.amazonaws.com/<repository>:<tag>
```

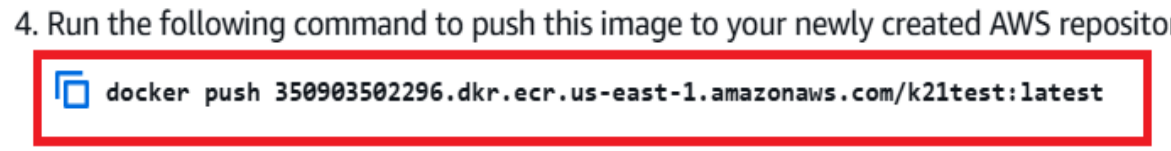
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

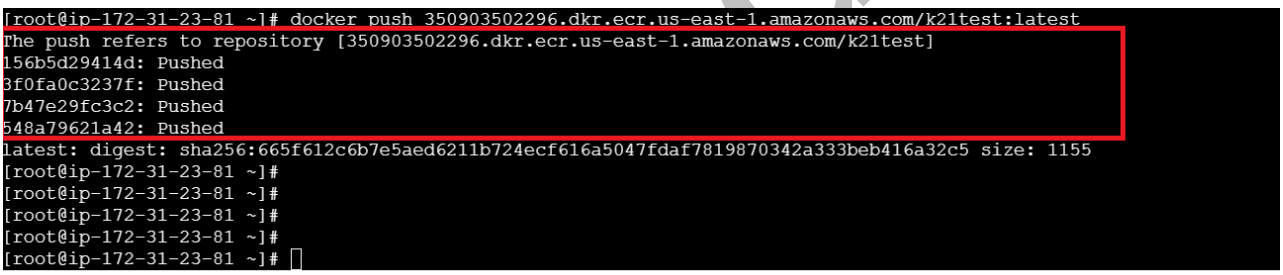
```
docker push 350903502296.dkr.ecr.us-east-1.amazonaws.com/k21test:latest
```

*The **Next Step is to push the image to ECR.***

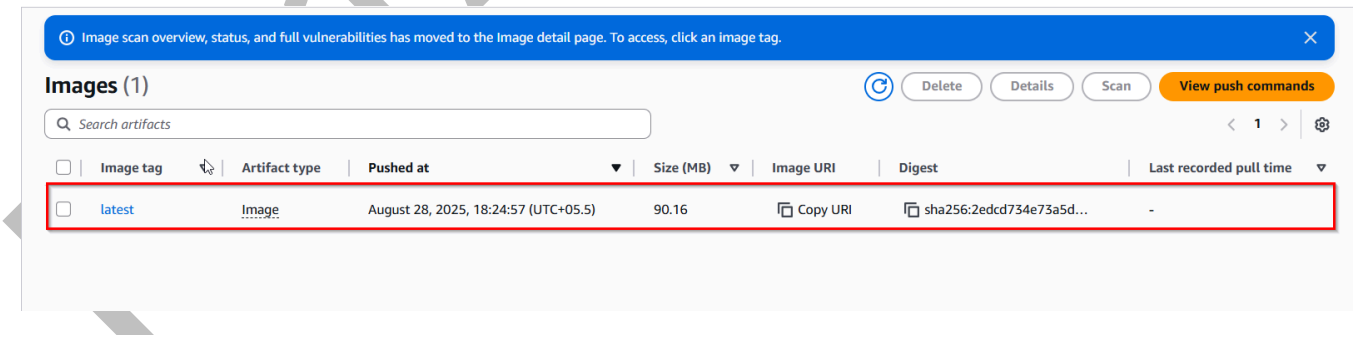
5. Copy & run the **Docker push** command, and the image will start uploading to the ECR repository.

NOTE: While pushing the Docker Image, you must use the root user on the Ubuntu server.





6. Back on the ECR console, your image will be successfully pushed to the Repository.



Now we have successfully pushed the Docker image to the ECR repository.

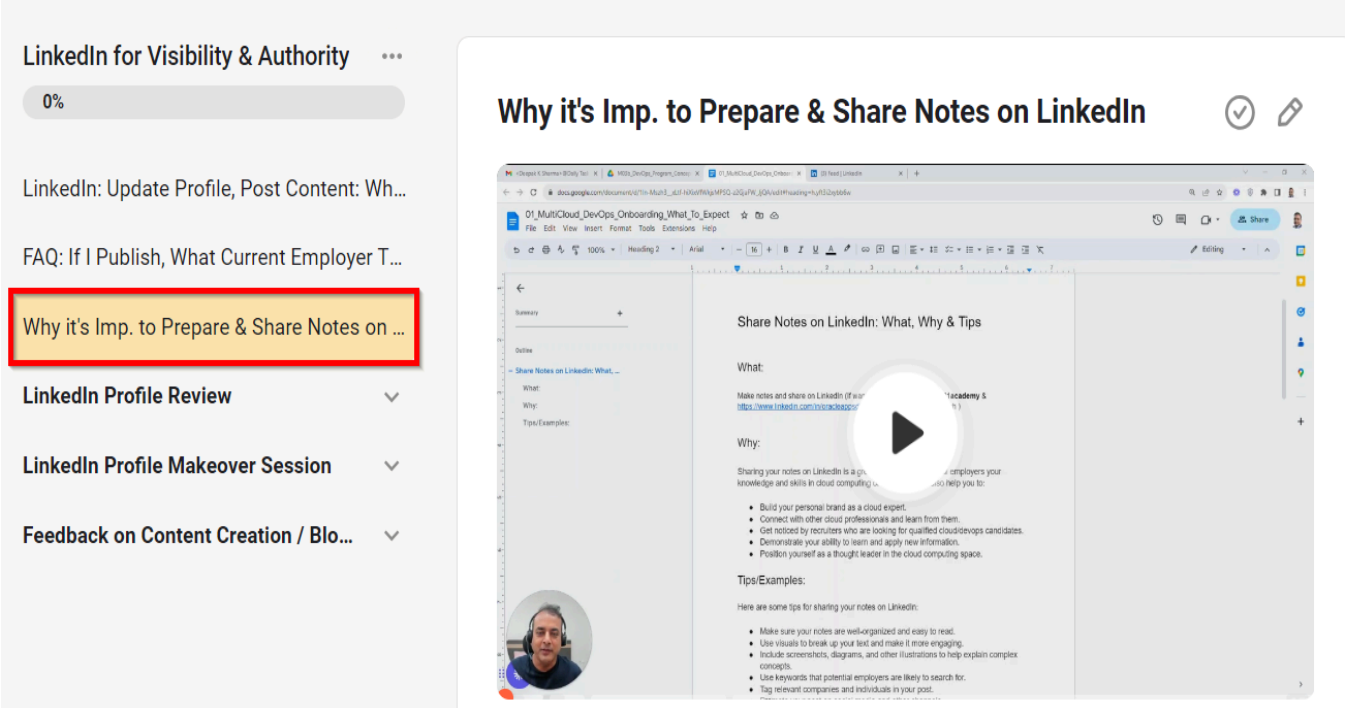
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

10 SHARE YOUR LEARNINGS

In this section, you will learn how to share whatever you learned during this lab on LinkedIn and WhatsApp.

Please watch the below👇 video below to understand why it's important to post & how to post:

https://www.skool.com/k21academy/classroom/0c24b6af?md=5d69343d4b5249ba8221c67b362cac6c



👩💻 As you know, building a strong professional profile is essential in today's job market. One way to showcase your skills and knowledge is by sharing your labs and projects on LinkedIn.

Your LinkedIn profile is a powerful tool in your job search. Sharing your labs and projects is just one way to demonstrate your expertise and stand out to potential employers. If you don't have LinkedIn, we strongly recommend you create one for yourself.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

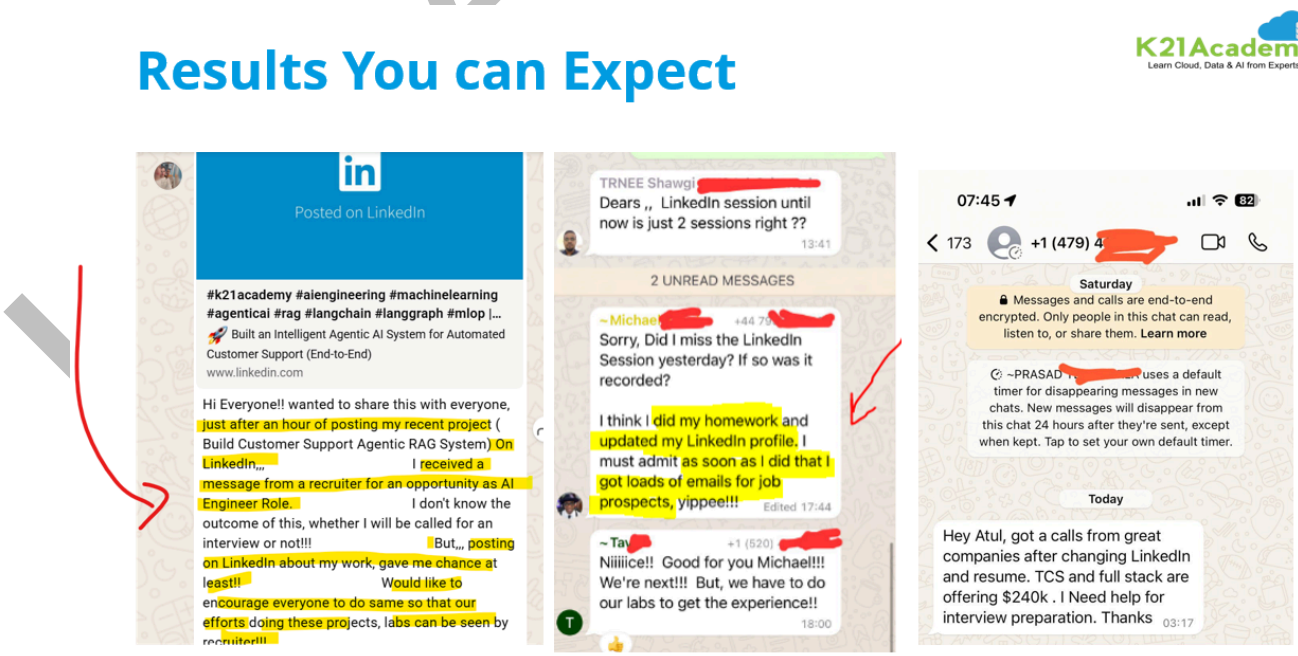
📝 🖥️ By doing so, you can demonstrate to potential employers or connections that you have hands-on experience in your field and are actively engaged in learning and growing your skills.

10.1 Share Hands-On Lab on LinkedIn to Build Authority & Job Opportunities

This will attract recruiters and potential employers to your profile and increase your Authority & LinkedIn reach.

1. **Do remember to tag so we can share too, and we'll ask other members to share too (this will increase your post reach)**
   a. K21Academy (https://www.linkedin.com/company/k21academy) &
   b. Atul Kumar (https://www.linkedin.com/in/atulk21academy/), as we will further circulate in our network, too, to increase your reach.

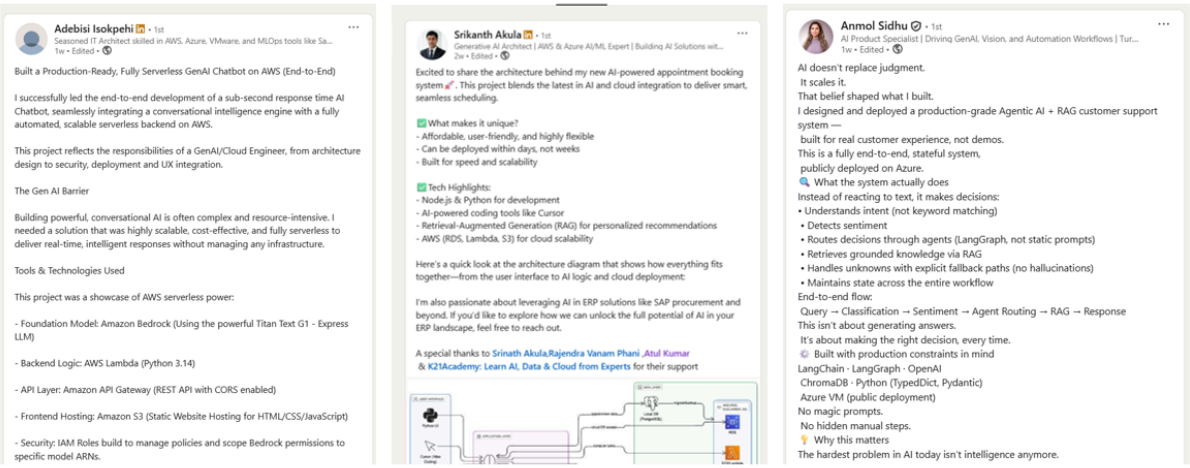
2. Here is what you can expect
   a. Jessy got a message from a recruiter for an AI Engineer Role
   b. Michael got loads of emails for Job prospects
   c. Prasad got calls from great companies that are offering $240K Salary



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

3. Example Links on How to post
    a. Example Posts

Post on LinkedIn for Interview Calls



Adebisi Isokpehi • 1st
Seasoned IT Architect skilled in AWS, Azure, VMware, and MLOps tools like Sa...
1w • Edited •

Built a Production-Ready, Fully Serverless GenAI Chatbot on AWS (End-to-End)

I successfully led the end-to-end development of a sub-second response time AI Chatbot, seamlessly integrating a conversational intelligence engine with a fully automated, scalable serverless backend on AWS.

This project reflects the responsibilities of a GenAI/Cloud Engineer, from architecture design to security, deployment and UX integration.

The Gen AI Barrier

Building powerful, conversational AI is often complex and resource-intensive. I needed a solution that was highly scalable, cost-effective, and fully serverless to deliver real-time, intelligent responses without managing any infrastructure.

Tools & Technologies Used

This project was a showcase of AWS serverless power:

- Foundation Model: Amazon Bedrock (Using the powerful Titan Text G1 - Express LLM)

- Backend Logic: AWS Lambda (Python 3.14)

- API Layer: Amazon API Gateway (REST API with CORS enabled)

- Frontend Hosting: Amazon S3 (Static Website Hosting for HTML/CSS/JavaScript)

- Security: IAM Roles build to manage policies and scope Bedrock permissions to specific model ARNs.

Srikanth Akula • 1st
Generative AI Architect | AWS & Azure AI/ML Expert | Building AI Solutions wit...
2w • Edited •

Excited to share the architecture behind my new AI-powered appointment booking system. This project blends the latest in AI and cloud integration to deliver smart, seamless scheduling.

What makes it unique?
- Affordable, user-friendly, and highly flexible
- Can be deployed within days, not weeks
- Built for speed and scalability

Tech Highlights:
- Node.js & Python for development
- AI-powered coding tools like Cursor
- Retrieval-Augmented Generation (RAG) for personalized recommendations
- AWS (RDS, Lambda, S3) for cloud scalability

Here's a quick look at the architecture diagram that shows how everything fits together—from the user interface to AI logic and cloud deployment:

I'm also passionate about leveraging AI in ERP solutions like SAP procurement and beyond. If you'd like to explore how we can unlock the full potential of AI in your ERP landscape, feel free to reach out.

A special thanks to Srinath Akula,Rajendra Vanam Phani ,Atul Kumar & K21Academy: Learn AI, Data & Cloud from Experts for their support

Anmol Sidhu • 1st
AI Product Specialist | Driving GenAI, Vision, and Automation Workflows | Tur...
1w • Edited •

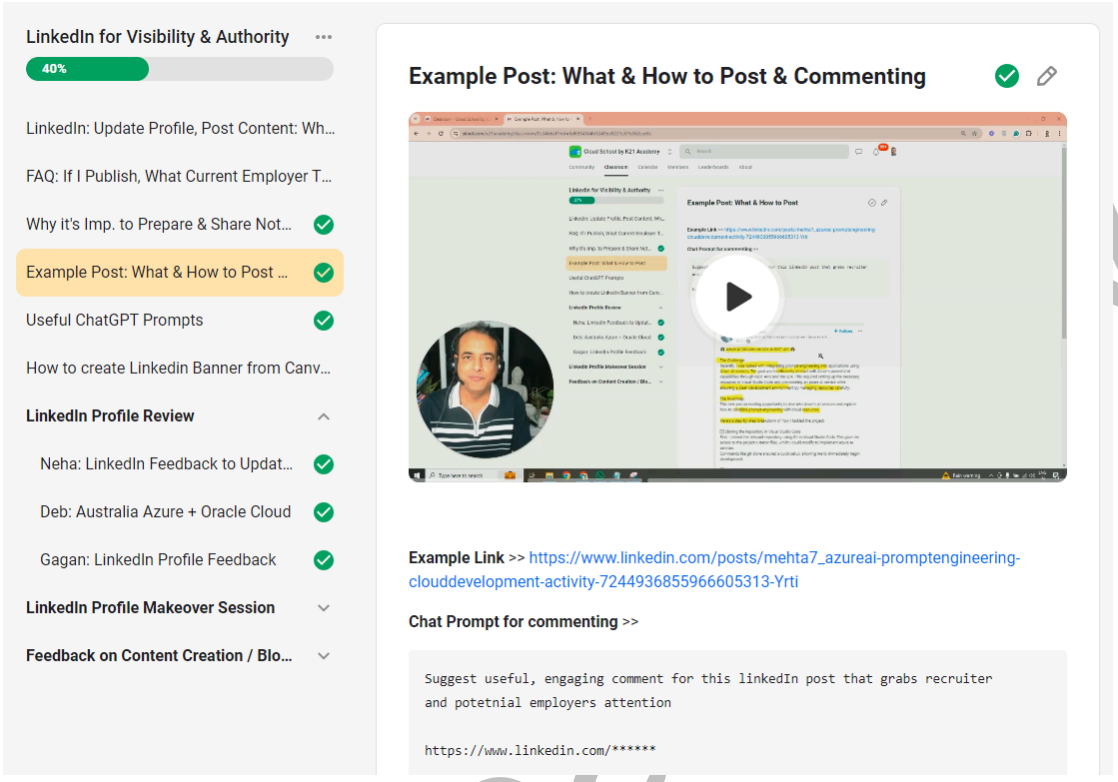
AI doesn't replace judgment.
It scales it.
That belief shaped what I built.
I designed and deployed a production-grade Agentic AI + RAG customer support system —
built for real customer experience, not demos.
This is a fully end-to-end, stateful system,
publicly deployed on Azure.
What the system actually does
Instead of reacting to text, it makes decisions:
• Understands intent (not keyword matching)
• Detects sentiment
• Routes decisions through agents (LangGraph, not static prompts)
• Retrieves grounded knowledge via RAG
• Handles unknowns with explicit fallback paths (no hallucinations)
• Maintains state across the entire workflow
End-to-end flow:
Query → Classification → Sentiment → Agent Routing → RAG → Response
This isn't about generating answers.
It's about making the right decision, every time.
Built with production constraints in mind
LangChain · LangGraph · OpenAI
ChromaDB · Python (TypedDict, Pydantic)
Azure VM (public deployment)
No magic prompts.
No hidden manual steps.
Why this matters
The hardest problem in AI today isn't intelligence anymore.

    b. Example Video & Post

K21Ac

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

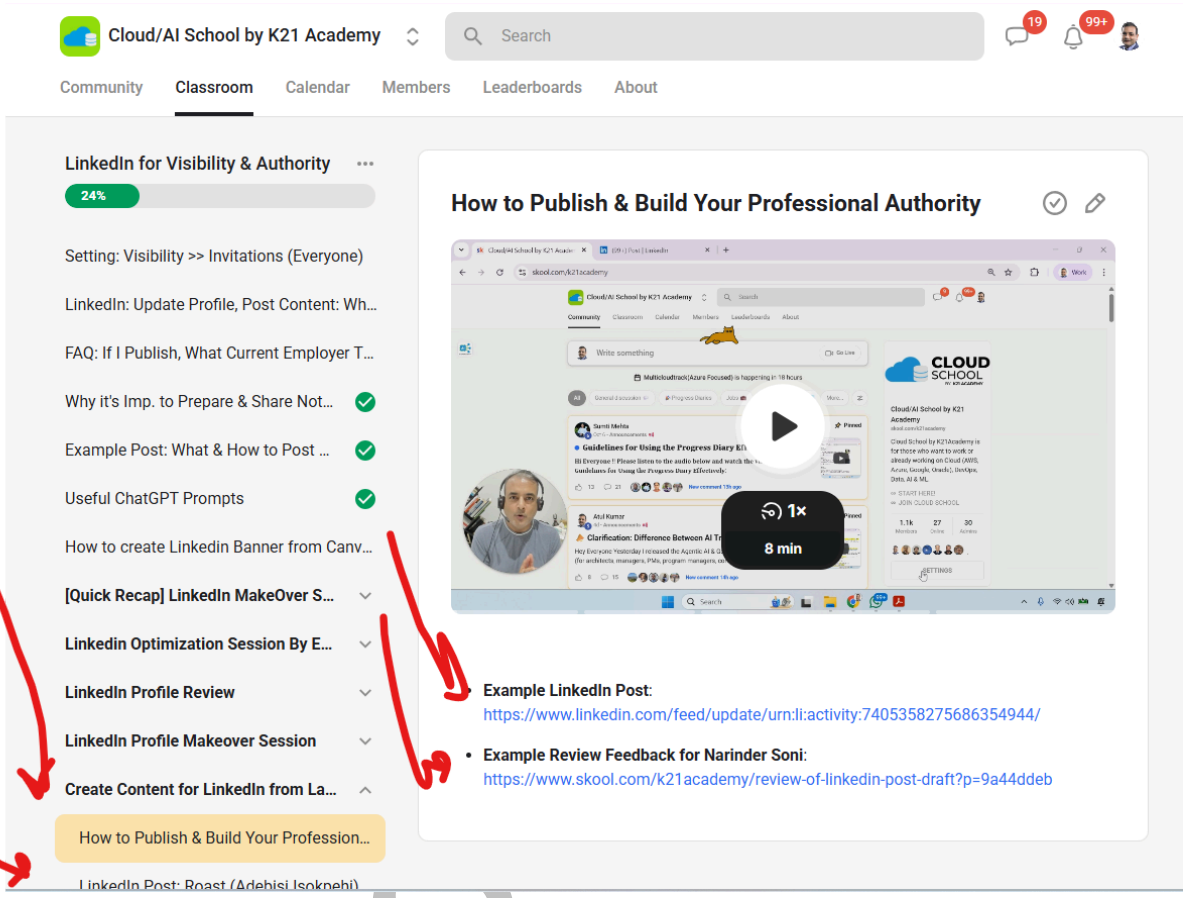
https://www.skool.com/k21academy/classroom/0c24b6af



c. Check few more linkedIn post examples (do's and don'ts): https://www.skool.com/k21academy/classroom/0c24b6af?md=edfe6092fbb84f6bba03ceb439d1e00a

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



10.2 Share the win with the Community

Share screenshots in Cloud School Community under your **'Progress Diary'**

If you don't know How to Access Your Progress Diary on Skool Portal check this:

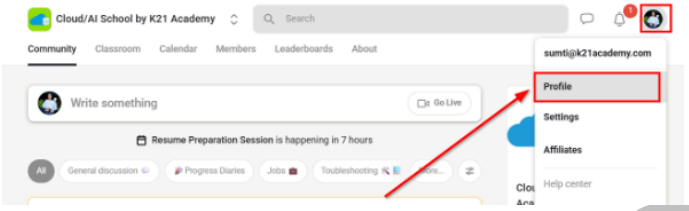
http://k21academy.com/FindYourDiary

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

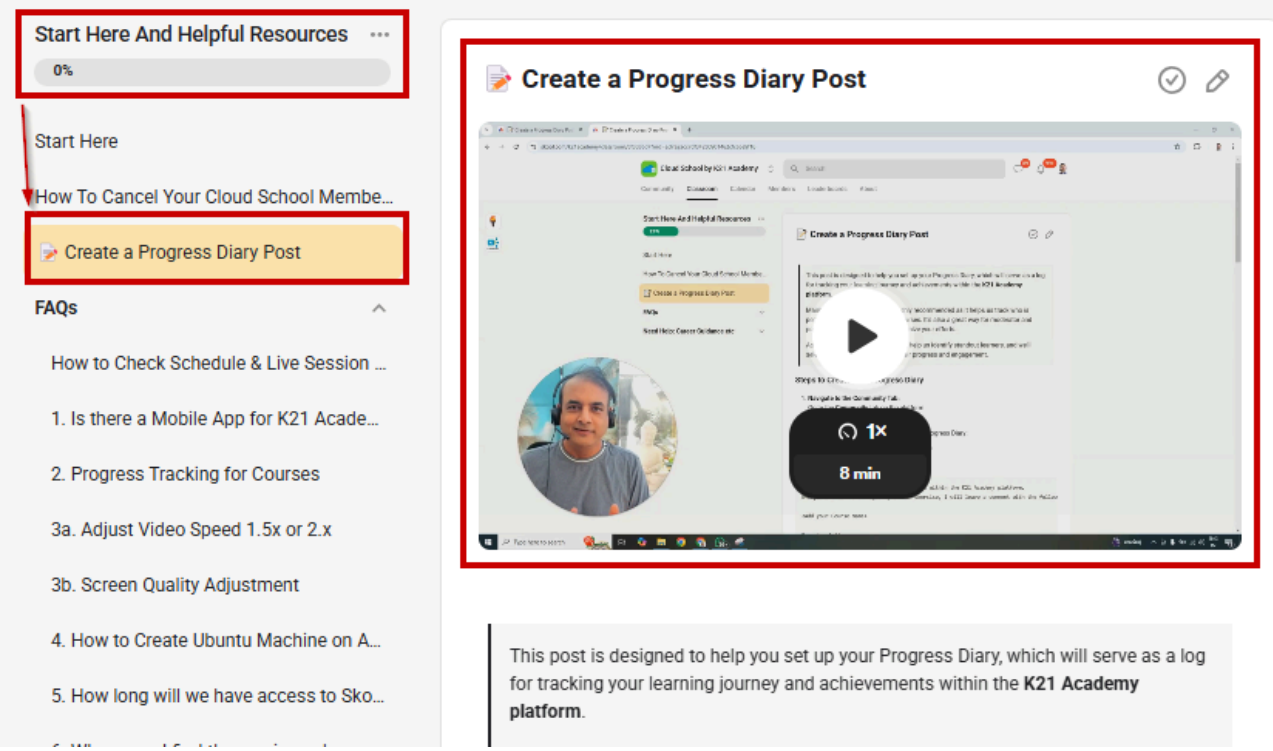
How to Access Your Progress Diary on Skool Portal

How to Access and Manage Your Progress Diary on Skool Portal

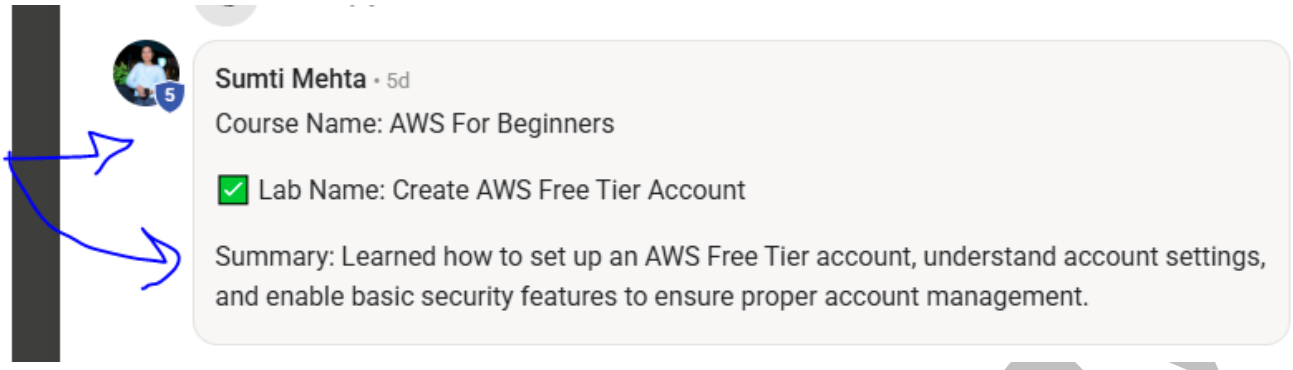
1. **Log In to Skool Portal:**
   - Go to the **Skool** website (or the link provided to you https://www.skool.com/k21academy).
   - Log in with your credentials (email/username and password).
2. **Navigate to Your Profile:**
   - Once logged in, click on your **profile picture** or **name** at the top right corner of the screen.



This will boost your confidence in the progressive path you are following, and encourage/inspire others to perform these hands-on labs.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



K21Academy

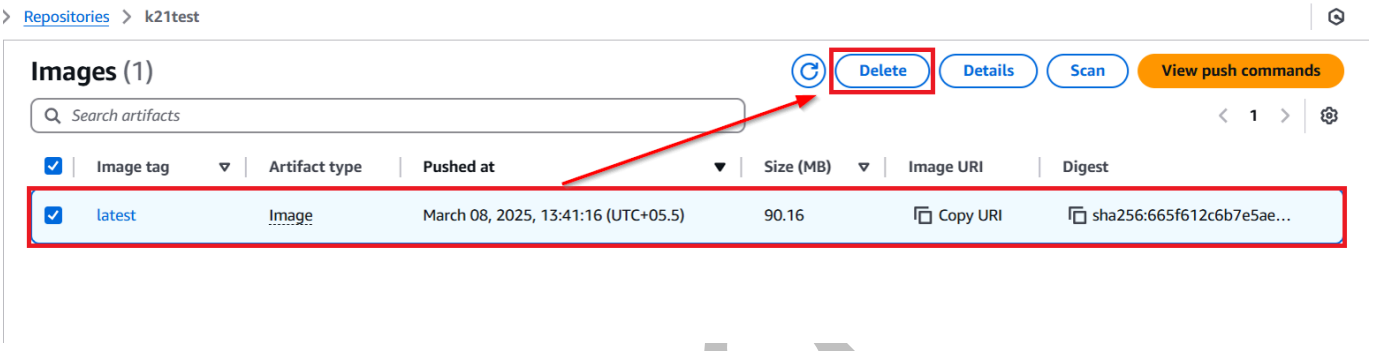
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

11 Cleanup Resources

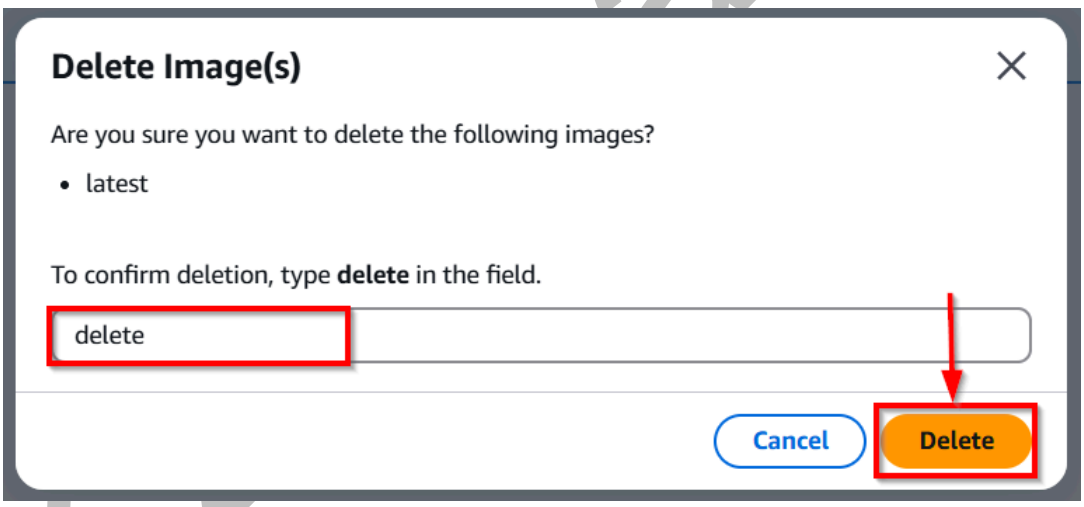
In this section, we will be deleting all the resources created by us so that we can avoid unnecessary charges.

11.1 Delete the pushed image and ECR

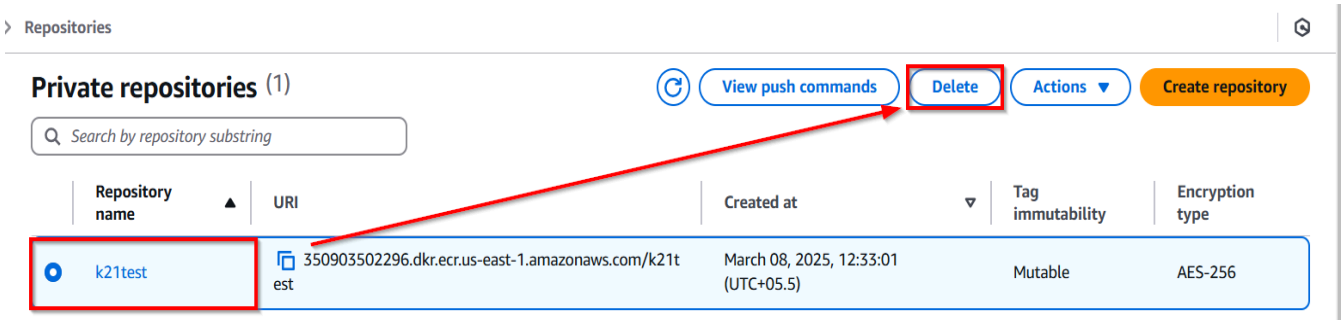
1. Back on the ECR console, delete the pushed image first

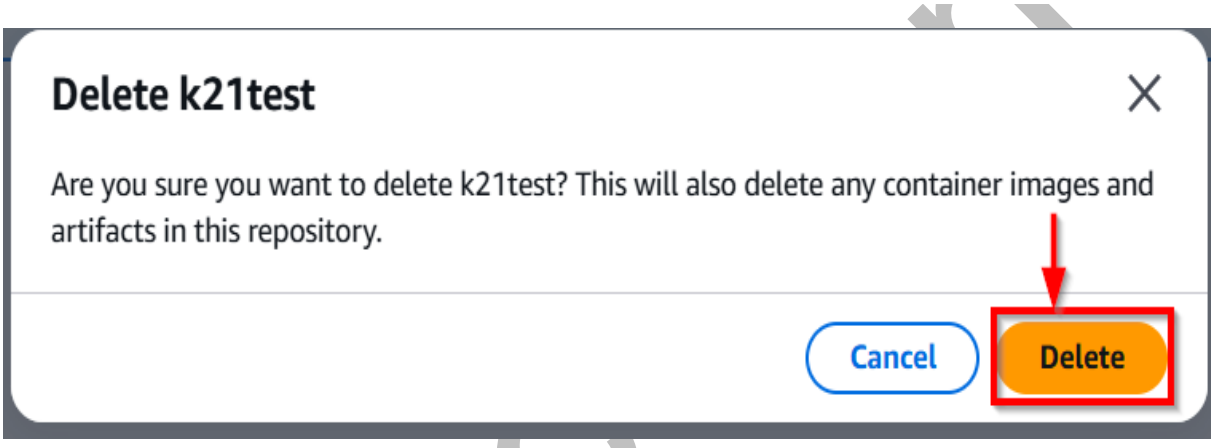


2. Type delete and click on Delete.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission





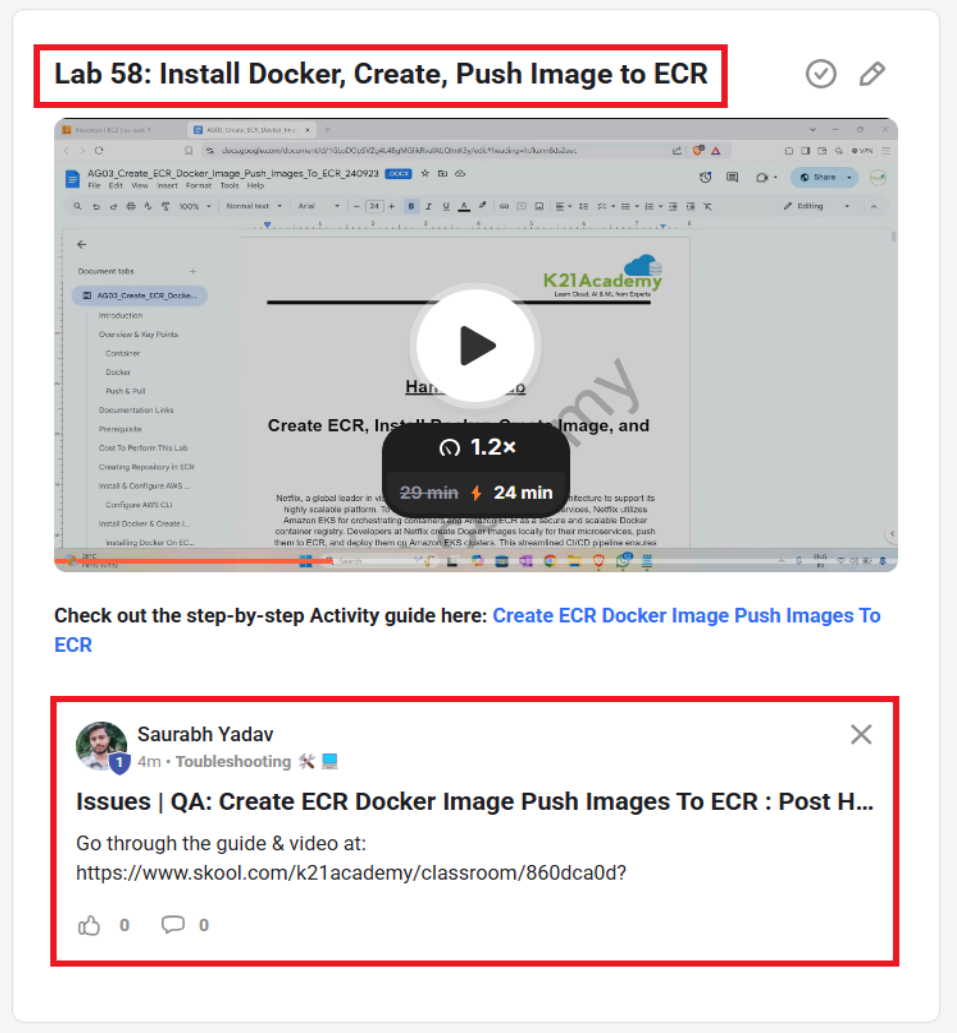
Note: **Make sure to delete the instance you have created.**

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

12 TROUBLESHOOTING

In this section, we outline some common issues you may encounter during the process, along with explanations of why these problems may occur. This will help you quickly identify and resolve any challenges you face.

Note: If you hit any issue in performing this lab, then add your issue in the thread below this lab (linked to this thread in the community)
https://www.skool.com/k21academy/issues-qa-create-ecr-docker-image-push-images-to-ecr-post-here

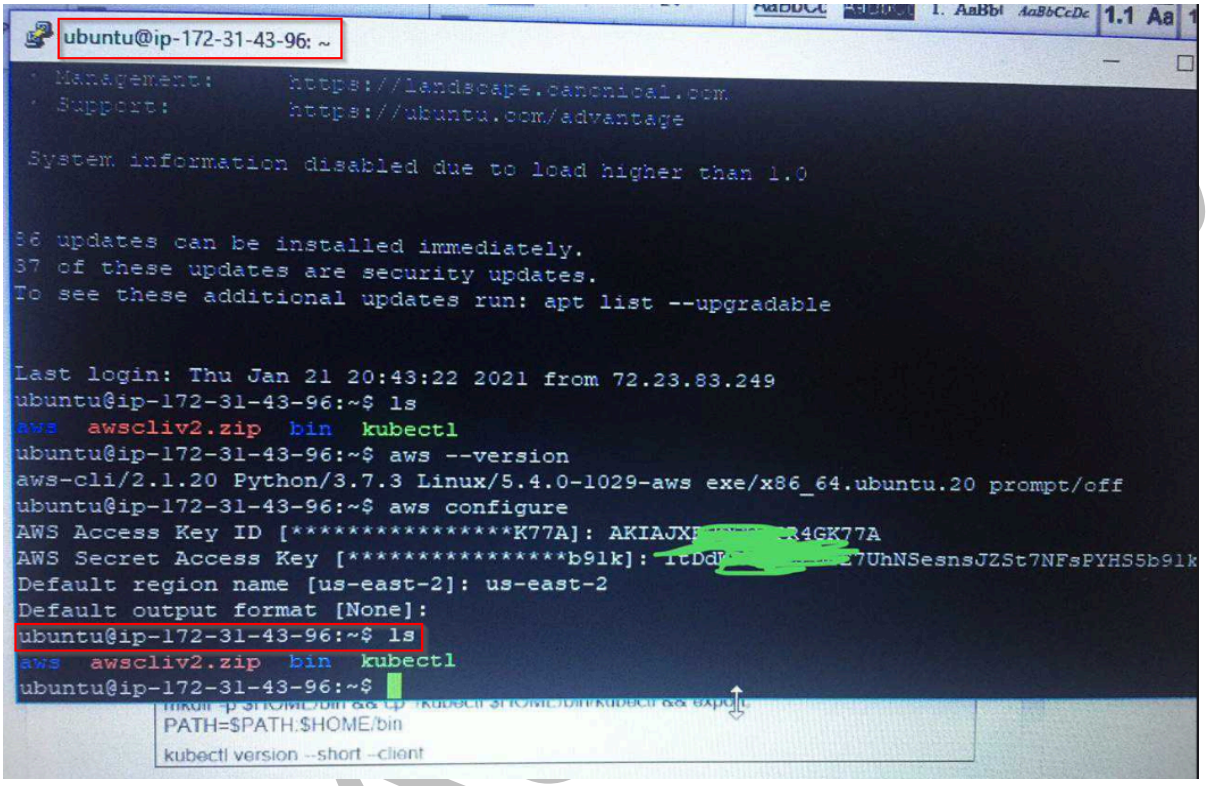


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

12.1 Unable to locate credentials

12.1.1 Issue

While pushing the Docker image, we faced such an issue.



12.1.2 Cause

The issue occurs due to the user we are using on the Ubuntu server; the Ubuntu user doesn't have access to the Docker files, and the Docker files are only accessed by the root user.

12.1.3 Fix

Before pushing the Docker image on the Ubuntu server, make sure you switch to the root user by running the command below.

```
$ sudo -i
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

13 SUMMARY

This activity guide covered steps to:

- Pushing Images to Amazon ECR
- Creating a Repository in ECR
- Configure AWS credentials and install Docker
- Creating a Docker file and building Images from it
- Pushing images to the Repository

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission